

AI Usage Log – Sprint 1

Project: SOEN 341 – CareerConnect (Job Search and Application Tracking Platform)

Team member: Lounis Benhamouche

AI tool used: Claude Code (Claude desktop app, Code tab), model Claude Opus 5.5

Period covered: October 1 – October 7, 2026

Summary of AI-Assisted Activities

#	Task	Date	Decision
1	Build the Job Seeker Profile page (User Profile Management)	October 1, 2026	Accepted
2	Debug "Cannot find module 'bcrypt'" and missing .env configuration	October 1, 2026	Accepted
3	Set up the local PostgreSQL database and resolve a port 3000 conflict	October 7, 2026	Modified before use

Task 1: Build the Job Seeker Profile page (User Profile Management)

Date: October 1, 2026

Task ID/Title	Task 1: Build the Job Seeker Profile page (User Profile Management)
Purpose of AI Use	Code generation (front-end implementation of a Sprint 1 basic feature: User Profile Management).
Chat Link or Prompt/Response	Local Claude Code session "Job site profile page" (no shareable link). Full prompt and response in Appendix, A.1.
AI-Suggested Content	• Filled in the empty frontend/profile.html with a profile page that reuses auth.css (same look as the login and register pages): initials avatar, live-updating name/headline/location header, and a form with first name, last name, headline, phone and location (the five fields the server accepts at /api/profile). • Created frontend/css/profile.css for the layout auth.css does not cover (two-column rows, avatar circle) without changing auth.css. • Created frontend/js/profile.js in the style of register.js: loads the saved profile on page load, then creates it (POST) or updates it (PUT) on save. • Flagged three blockers for saving: no logged-in user ID is stored yet (localStorage.demoUserId), the server only enables CORS for /api/auth/register, and the server must run with DEMO_AUTH=true.
Validation	• The AI previewed the page in a browser at desktop and phone widths and tested the live-updating header. • Manual review of the generated HTML/CSS/JS against the existing auth.css, register.html and register.js conventions. • Checked that the form fields match the backend /api/profile endpoint in server.js. • Saving was NOT tested end-to-end, because the backend was not running at the time.
Decision	Accepted. (Review and update if the team modified it before merging.)
Reflection	The AI was useful for quickly producing a page consistent with the existing style and for spotting integration gaps (missing user ID storage, CORS scope, demo-auth flag) that would otherwise have surfaced later. The save flow still needs to be tested once login stores the user ID.
Responsible Person	[Your Name]

Task 2: Debug "Cannot find module 'bcrypt'" and missing .env configuration

Date: October 1, 2026

Task ID/Title	Task 2: Debug "Cannot find module 'bcrypt'" and missing .env configuration
Purpose of AI Use	Debugging and problem solving (local development environment setup).
Chat Link or Prompt/Response	Local Claude Code session "Missing bcrypt module" (no shareable link). Full prompts and responses in Appendix, A.2.
AI-Suggested Content	• Diagnosed that node_modules did not exist (dependencies never installed after cloning/pulling) and ran npm install. • Explained that npm's blocked bcrypt install script is harmless on macOS because bcrypt ships a prebuilt binary. • For the follow-up "DATABASE_URL is required" error, created .env from .env.example (confirmed git ignores it). • Explained that PostgreSQL must be running (via Docker, per the README), gave install and setup steps (docker compose up -d --wait db, npm run db:migrate, npm start), and noted that only the npm scripts load .env.
Validation	• Checked that bcrypt loads with a small Node test (hashSync). • Ran npm test: 1 passed, 0 failed. • Confirmed .env is listed in .gitignore so no local credentials are committed.
Decision	Accepted.
Reflection	The root cause was simple (dependencies not installed), but the AI also anticipated the next errors (.env missing, no database running). Lesson learned: after every pull that changes package.json, run npm install; start the app with npm start, not node server.js.
Responsible Person	[Your Name]

Task 3: Set up the local PostgreSQL database and resolve a port 3000 conflict

Date: October 7, 2026

Task ID/Title	Task 3: Set up the local PostgreSQL database and resolve a port 3000 conflict
Purpose of AI Use	Debugging and problem solving (environment setup, database migration, verification).
Chat Link or Prompt/Response	Local Claude Code session "Missing bcrypt module" (continued). Full prompts and responses in Appendix, A.3.

AI-Suggested Content	• Found that Docker Desktop was installed but not on the terminal PATH, started the Postgres container and ran the migrations. • Diagnosed that the health check failed because a separate PostgreSQL 18 install on the machine was listening on port 3000; confirmed the app itself worked by running it on port 3999. • Proposed two options: (A, recommended) move the separate Postgres to port 5433, or (B) move the app to another port, which would require changing the hard-coded URL in frontend/js/register.js (shared team code). • Explained that the npm 12 update notice was unrelated and that the migration had succeeded (exit code 0).
Validation	• I chose Option A and applied the change myself (it needed admin rights); the AI then re-verified everything: • Ports: separate Postgres on 5433, Docker database on 5432, port 3000 free for the app. • GET http://localhost:3000/api/health returned {"status":"ok","database":"connected"}. • A test registration reached the database and returned "User registered successfully" (this also exercises bcrypt). • npm test: 1 passed, 0 failed.
Decision	Modified before use: the recommended Option A was chosen and applied manually by me; Option B was rejected so that shared team code (register.js) did not need to change.
Reflection	Very useful: the port conflict was not obvious, since the error was just an empty response. Testing on an alternate port isolated the problem quickly. Note: the verification left a test account (verify2214@example.com) in the local database. Follow-up idea for the team: replace the hard-coded http://localhost:3000 URL in register.js with a relative URL.
Responsible Person	[Your Name]

Appendix – Prompts and AI Responses

The AI conversations were held in local Claude Code sessions, which have no shareable link, so the prompts and the relevant AI responses are reproduced below. Long error stack traces are shortened to their key lines. Tool actions taken by the AI (reading files, running commands) are summarized in brackets.

A.1 – Task 1: Profile page (October 1, 2026)

Prompt

```
@frontend/css/auth.css
Create a profile page for a job finding website and respect the style of this css file
```

AI response

[AI inspected frontend/, auth.css, register.html, register.js and server.js; wrote profile.html, profile.css and profile.js; previewed the page in a browser at desktop and mobile widths.]

I've built the profile page. It uses auth.css as its base stylesheet, so it looks like the login and register pages: the same dark red gradient background, thin white lines above and below the content, spaced-out "CareerConnect" title, underline-only inputs and dark red uppercase button. I checked it in the browser at desktop and phone widths and it looks right. I haven't tested saving, because the backend wasn't running.

What's on the page

- Header: a circle with the user's initials, plus their name, headline and location. It updates as they type.
- Form fields: first name, last name, headline, phone and location. These are the five fields your server already accepts at /api/profile. Names and phone/location sit side by side on desktop and stack on phones.
- Message line: it shows errors, and shows confirmations in a lighter colour.

Files

- frontend/profile.html: this was already in the repo but empty, so I filled it in.
- frontend/css/profile.css: a small new file for the parts auth.css doesn't cover, like the two-column rows and the initials circle. I didn't change auth.css.
- frontend/js/profile.js: written in the same style as register.js. On page load it fetches the user's saved profile. On save it creates the profile if there isn't one yet, otherwise it updates it.

Before saving will work

- No logged-in user to send. Until real login exists, your server identifies the user by a user ID sent in an x-demo-user-id header. The page reads that ID from localStorage.demoUserId, but nothing stores it yet. Until it's set, the page shows "Please log in to view your profile." One fix is to have register.js save the user.id that the register request returns.
- The server blocks the profile requests from the browser. Its cross-origin setting (cors()) only covers /api/auth/register. Unless the page is served from the same address as the API, the browser will block the profile requests until you add cors() for /api/profile too.
- Demo mode must be on. The server only accepts that user-ID header when it's started with DEMO_AUTH=true.

The "Browse Jobs" link at the bottom points to # for now, because there's no jobs page yet.

A.2 – Task 2: bcrypt and .env errors (October 1, 2026)

Prompt 1

```
Error: Cannot find module 'bcrypt'
Require stack:
- /Users/.../soen-341/server.js
  at Module._resolveFilename (node:internal/modules/cjs/loader:1591:15)
  ...
  code: 'MODULE_NOT_FOUND'
```

AI response 1

[AI checked package.json and found no node_modules folder, ran npm install, verified bcrypt loads, ran npm test.]

I fixed it. The dependencies had never been installed: bcrypt was in package.json, but there was no node_modules folder. That usually happens after a fresh clone, or when a teammate adds a package and you pull their change. I ran npm install, and now bcrypt loads and npm test passes (1/1).

npm warned that bcrypt's install script was blocked. That's fine on your Mac, because bcrypt ships a ready-made build for macOS and uses it automatically.

One more thing: you don't have a .env file yet, only .env.example. npm start runs node --env-file=.env, so it will fail until you create one. Copy the example (cp .env.example .env), edit it to point at your database, then run npm start. Whenever someone adds a new dependency, run npm install again after you pull.

Prompt 2

```
/Users/.../soen-341/db/index.js:3
  if (!process.env.DATABASE_URL) throw new Error('DATABASE_URL is required. Copy
.env.example to .env and configure PostgreSQL.');
```

Error: DATABASE_URL is required. Copy .env.example to .env and configure PostgreSQL.
at createPool (/Users/.../soen-341/db/index.js:3:40)
...

AI response 2

[AI created .env from .env.example and confirmed git ignores it; read compose.yaml and the README.]

I created .env from .env.example, which fixes the DATABASE_URL is required error. Git ignores the file, so it won't be committed.

Next problem: if you run npm start now, it will fail with a connection error (ECONNREFUSED). The app needs a PostgreSQL database, and you don't have one running. The README's setup runs Postgres in Docker, but Docker isn't installed on your Mac.

- Install Docker Desktop, then open it once so it finishes starting.
- Start the database: docker compose up -d --wait db (compose.yaml uses the same username and password as .env).
- Create the tables: npm run db:migrate
- Start the server: npm start, then open <http://localhost:3000/api/health> to check it is working.

Run npm start (or npm run db:migrate), not node server.js. Only the npm scripts load .env. Once Docker is installed, I can run steps 2–4 for you.

A.3 – Task 3: Database setup and port conflict (October 7, 2026)

Prompt 1

```
$ docker compose up -d --wait db
zsh: command not found: docker
```

AI response 1

[AI found Docker Desktop installed under ~/.docker/bin, started the Postgres container, ran migrations, started the server and investigated why the health check returned nothing: tested the app on port 3999 and identified the process holding port 3000.]

Docker, the database, and the migrations are all working now. The last problem is a port conflict, and the fix is your call.

What's happening: a separate PostgreSQL 18 install on your Mac (in /Library/PostgreSQL/18) is set to use port 3000, the same port as the app. When you visit http://localhost:3000, Postgres answers instead of the app and drops the connection. I started the app on port 3999 as a test, and it returned {"status":"ok","database":"connected"}, so the app itself is fine. frontend/js/register.js:50 sends registrations to http://localhost:3000, so moving the app to another port would break sign-up unless that file changes too.

- **Option A (recommended):** move the separate Postgres off port 3000 (to 5433) by editing /Library/PostgreSQL/18/data/postgresql.conf and restarting it with pg_ctl. This needs your admin password, so run these yourself.
- **Option B:** run the app on another port (e.g. PORT=3001 in .env). This means changing the hardcoded URL in register.js, which is shared team code; the proper fix would be a relative URL (/api/auth/register).

Which option do you want? If you pick B, I can make the register.js change.

Prompt 2

```
$ npm run db:migrate
> soen-341@1.0.0 db:migrate
> node --env-file=.env scripts/migrate.js
npm notice New major version of npm available! 11.19.1 -> 12.2.0
```

AI response 2

That migration run worked. It exits with code 0, which only happens when the migrations applied cleanly. The "Database migrations applied." line was probably pushed aside by npm's update notice. The migrations are safe to re-run. You can ignore the npm 12 notice. The only thing still blocking npm start is the port 3000 conflict.

Prompt 3

```
i did option A verify now if everything works
```

AI response 3

[AI checked listening ports, started the server, called the health endpoint, sent a test registration and ran npm test.]

Everything works now. I checked each piece:

- Ports: your separate Postgres is now on 5433, the project's Docker database is on 5432, and nothing else is using port 3000.
- App: http://localhost:3000/api/health returns {"status":"ok","database":"connected"}.
- Registration: a sign-up request reached the database and returned "User registered successfully". It also uses bcrypt, the package that caused your first error.
- Tests: npm test passes (1 passed, 0 failed).

My check left a test account, `verify2214@example.com`, in your local database. From now on, start the project with `docker compose up -d --wait db` and then `npm start`. After a restart, open Docker Desktop first. You only need `npm run db:migrate` again when someone adds a new migration.